智检 Agent —— 工业表面缺陷智能检测与处置智能体系统

软件设计文档

项目	内容
作品名称	智检 Agent —— 面向中小制造企业的工业表面缺陷智能检测与处置智能体系统
参赛赛道	赛道一：人工智能在垂直应用场景的创新设计与开发（智能技术应用类）
赛事	2026 年华北五省（市、自治区）及港澳台大学生计算机应用大赛
作品版本	v1.0.0
文档版本	v1.0

目录

1. 作品概述
2. 需求分析
3. 总体设计
4. 功能模块设计
5. 关键技术与实现
6. 数据库设计
7. 系统测试与指标
8. 部署与操作说明
9. 创新点与技术特色
10. 开发规范与工具说明

---

1 作品概述

1.1 作品简介

"智检 Agent" 是一套面向中小制造企业的**工业表面缺陷智能检测与处置智能体系统。**它把一次质检拆成五个连续环节并全部自动化：

> 拍照/上传 → 缺陷检测与定位 → 严重度分级 → 智能体根因分析与工艺处置 → 工单与报告闭环

系统在一台普通笔记本电脑（无独立显卡）上即可完成训练与实时检测，单张图像检测耗时约 0.3~0.8 秒，整机不含任何专用视觉硬件。

1.2 解决的问题

中小企业表面质检面临三重矛盾：

1. **人力矛盾**：质检岗位依赖人眼，连续工作 8 小时后漏检率显著上升，且招工日益困难。
2. **成本矛盾**：商用机器视觉整机方案单工位投入高，中小企业难以承担。
3. **数据矛盾**：工厂带宽有限、工艺数据敏感，"数据不出厂"是硬约束，纯云端方案难以落地。

更关键的是，现有低成本方案普遍**只输出"是什么缺陷"，不输出"该怎么办"**。产线真正需要的是处置决策：根因在哪、调什么参数、要不要停机、同批次怎么办。本作品的智能体正是补齐这一环。

1.3 主要功能

序号	功能	说明
1	缺陷检测	多尺度滑窗 + 轻量 CNN，检出六类表面缺陷并框出位置
2	严重度分级	类别风险 × 置信度 × 面积占比 → A/B/C/D/E 五级与处置动作
3	可解释性	Grad-CAM 热力图输出模型关注区域，支持人工复核
4	处置智能体	检索标准知识库，生成根因分析、立即处置、工艺纠正、批次追溯建议
5	工单闭环	按缺陷类型自动生成维修/工艺工单，联动责任部门与检查项
6	质量看板	合格率、等级分布、缺陷类型分布、逐小时趋势、节拍统计
7	报告输出	一键生成 Markdown / HTML（可打印为 PDF）/ JSON 质检报告
8	知识问答	面向质检员的自然语言问答，结论可溯源到知识库条目

2 需求分析

2.1 政策依据

文件	文号	与本作品的对应关系
《国务院关于深入实施"人工智能+"行动的意见》	国发〔2025〕11号	提出"推进工业全要素智能化发展，增强工业系统的智能感知与决策执行能力"——本作品以机器视觉实现感知、以智能体实现决策执行
《"人工智能+制造"专项行动实施意见》	工信部联科〔2025〕279号	明确"推广机器视觉、无人智能巡检等工业质检技术"；质量检测场景"重点开展基于预训练模型的小样本标注缺陷数据微调"；到2027 年推出 1000 个高水平工业智能体
《中小企业数字化赋能专项行动方案（2025—2027）》	工信部联企业〔2024〕239号	提出培育挖掘"视觉质量检测"等典型场景
中小企业数字化转型城市试点（第三批）	财办建〔2025〕20号	提出"因地制宜推进人工智能大模型……在视觉质检等场景中的应用"
《中共中央关于制定国民经济和社会发展第十五个五年规划的建议》	—	"全面实施'人工智能+'行动……全方位赋能千行百业"

执行标准：GB/T 40659-2021《智能制造 机器视觉在线检测系统 通用要求》、GB/T 45575-2025《工业产品表面缺陷自动检测系统技术要求》、GB/T 41867-2022《信息技术 人工智能 术语》、GB/T 32738-2016《热轧钢带表面质量》。

> 注：上述标准号均经全国标准信息公共服务平台核对。本作品不对未公开全文或文号存疑的文件作引用。

2.2 用户角色与场景

角色	关注点	系统提供
产线质检员	这件合不合格？该不该拦？	明确的判定结论、等级与建议动作
工艺工程师	缺陷从哪来？调什么参数？	根因分析、工艺纠正措施、标准依据
设备科	要不要停机检修？查什么？	自动工单、责任部门、检查项清单
质量主管	这批能不能放？趋势是否恶化？	批次追溯建议、看板趋势与分布
企业管理者	值不值得上？投入产出如何？	合格率、节拍、缺陷构成等量化指标

2.3 功能需求

以"高内聚低耦合"为原则，划分为 **感知层、决策层、应用层** 三层，共 8 个功能模块（见第 3、4 章）。

2.4 非功能需求

类别	要求	本作品达成方式
性能	单张检测 ≤ 1 s	轻量主干 + 滑窗批量推理，实测见第 7 章
部署成本	无需专用视觉硬件与 GPU	全 CPU 可训练可推理，普通笔记本运行
数据安全	数据不出厂	默认离线运行；本地大模型后端可选，不外传图像
可解释性	判定可追溯、可复核	Grad-CAM 热力图 + 知识库引用 + 全链路留痕
可移植性	不依赖联网与外部服务	核心功能零额外依赖（仅 torch/opencv/fastapi 等基础库）
可维护性	参数可配置	全部关键参数集中于 `config/config.yaml`

---

3 总体设计

3.1 系统架构

```
┌──────────────────────────────────────────────────────────────┐
│  应用层   Web 前端（检测 / 看板 / 记录 / 工单 / 知识问答 / 说明）      │
│           FastAPI REST 接口 + 质检报告生成（MD / HTML / JSON）        │
├──────────────────────────────────────────────────────────────┤
│  决策层   质检处置智能体 QualityAgent                                 │
│           工具：retrieve_standard / severity_grade / create_work_order│
│                 trace_batch / self_check                          │
│           后端：rule（确定性）→ ollama（本地）→ openai_compat（云）     │
│           检索：BM25 + 字符 n-gram 混合（知识库 18 条专业条目）          │
├──────────────────────────────────────────────────────────────┤
│  感知层   图像预处理 → 多尺度滑窗 → MobileNetV3-Small 批量推理          │
│           → 按类 NMS → 严重度评分 → Grad-CAM 可解释性                 │
├──────────────────────────────────────────────────────────────┤
│  数据层   SQLite（检测记录 / 工单 / 检索留痕）                         │
│           数据集：程序化合成 + 公开真实数据（可扩展企业自有数据）          │
└──────────────────────────────────────────────────────────────┘
```

3.2 技术选型与理由

环节	选型	选择理由
检测主干	MobileNetV3-Small（1.08 M 参数）	CPU 单窗口推理 10~30 ms，满足无 GPU 环境；BSD-3 许可，便于声称可交付企业
定位方式	分类器 + 多尺度滑窗 + 按类 NMS	只需图片级标签即可训练，大幅降低标注门槛；对应政策"小样本标注缺陷数据微调"
可解释性	Grad-CAM（自研实现）	不引入额外依赖；满足在线检测系统可追溯与人工复核要求
知识检索	自研 BM25 + 字符 n-gram 混合	中文专业术语场景下字符 n-gram 召回优于词典分词；零外部依赖，演示机不联网也能跑
智能体后端	规则 / Ollama / OpenAI 兼容 API 三级降级	规则后端永远可用且确定性，大模型仅用于增强表达，避免幻觉影响质量判定
Web 框架	FastAPI + 原生 HTML/CSS/JS	无前端构建链路，单机双击即用；评委可离线打开
存储	Python 标准库 sqlite3	零安装、单文件、便于随作品打包与人工查看
模型导出	ONNX（可选）	便于部署到无 PyTorch 环境的工控机，规避框架依赖冲突

3.3 关键设计决策

决策一：不把大模型放在质量判定路径上。

放行/判废由确定性的规则与阈值完成，大模型只负责生成解释与建议。理由是工业质量判定必须可复现、可审计，不能受模型随机性影响。

决策二：定位用滑窗而非端到端检测器。

本项目一度考虑 YOLO 系列，最终放弃，原因有三：其一，YOLO 采用 AGPL-3.0，官方许可页明确将"自训练/微调的权重"也纳入约束，不利于声称可交付企业；其二，端到端检测需要框级标注，小厂不具备条件；其三，滑窗分类在 CPU 上的工程复杂度更低、可解释性更强。

决策三：三级后端降级。

比赛演示现场可能无网络、无本地大模型。规则后端保证"任何环境都能演示完整链路"，这是工程可靠性的底线。

---

4 功能模块设计

4.1 数据模块

模块	文件	职责
合成数据生成	`src/data_generate.py`	程序化合成六类带钢缺陷与洁净样本，参数化可控、可复现
真实数据接入	`src/data_real.py`、`src/data_github.py`	下载整理公开真实缺陷数据集，统一为 `normal/defect` 结构
数据索引	`src/data.py`	分层抽样划分、归一化统计量估计、PyTorch Dataset 封装
数据核验	`src/visualize_data.py`	类别拼图抽样，人工核验数据质量

合成缺陷的物理建模：基础纹理由轧制方向条纹 + 分形噪声 + 低频光照不均合成；六类缺陷按形成机理分别建模——龟裂为晶界网状裂纹，夹杂为深色不规则块状压入物，斑块为片状氧化区并抹除部分纹理，麻点为密集小凹坑，压入氧化铁皮为沿轧制方向的条带压痕并伴随细裂纹，划痕为细长线状机械损伤。

三层数据策略：合成数据解决冷启动 → 公开真实数据建立可信基线 → 企业自有数据微调落地。这一路线直接回应工业界"缺陷样本稀缺、冷启动无数据"的核心痛点。

4.2 模型模块

`src/model.py` 提供轻量缺陷分类器与 Grad-CAM：

- 主干：MobileNetV3-Small，参数量 1.08 M；
- 分类头：Dropout → Linear(576→256) → GELU → Dropout → Linear(256→7)；
- 输入：单通道灰度复制为 3 通道，兼容 ImageNet 预训练权重；
- 类别空间：6 类缺陷 + 1 类洁净 = 7 类；
- Grad-CAM：对主干最后卷积块做梯度加权，输出类激活热力图。

> 开发中记录了两个 torchvision 使用陷阱，已写入代码注释并加入自检脚本 `src/selfcheck_model.py`：① `mobilenet_v3_*` 直接调用 `net(x)` 返回的是**池化分类后的向量**而非特征图，必须取 `net.features`；② `resnet18` 的 `forward` 内部依赖 `avgpool` 才能正确 `flatten`，将 `avgpool` 置为 `Identity` 会导致特征维度错误。

4.3 训练模块

`src/train.py` 流程：加载索引 → 类别加权交叉熵 → AdamW + 余弦退火 → 早停 → 最优权重落盘 → 测试集评估 → 生成训练曲线与混淆矩阵 → 可选导出 ONNX。

评估指标包括：总体准确率、Top-2 准确率、宏平均/加权平均 P-R-F1、**期望校准误差 ECE**、逐类别 P/R/F1、混淆矩阵、单图推理耗时。其中 ECE 用于衡量置信度是否可信——在"低置信度转人工复检"的策略下，校准质量直接决定该策略是否有效。

4.4 推理模块

`src/infer.py` 实现滑窗检测：

1. 图像灰度化；
2. 多尺度滑窗切块（尺度 1.0 与 0.6，窗口 224，步长 112），兼顾小目标与大缺陷；
3. 批量前向（批大小 32，`torch.inference_mode()` + 多线程），取非洁净类别；
4. 依置信度阈值过滤；
5. 按类别执行 NMS（IoU 阈值 0.35）合并重叠窗口；
6. 计算面积占比并做严重度评分；
7. 输出结构化结果与可视化（标注图 + Grad-CAM 热力图）。

严重度评分公式（v2）：

```
severity = 100 · [ w_risk · R + (1 - w_risk) · ( w_conf · C + w_area · A ) ]


w_risk = 0.55, w_conf = 0.20, w_area = 0.25（后两者内部归一化后为 0.44 / 0.56）
R = 类别风险基线分 ∈ {high: 0.70, mid: 0.42, low: 0.18}
C = 检出置信度
A = min( min(1, sqrt(area / 0.05)), 0.30 )
```

分级映射：A ≤ 28 放行；B ≤ 42 放行并记录；C ≤ 60 人工复检；D ≤ 80 判废并停机排查；E ≤ 100 紧急停机并隔离批次。

4.4.1 分级公式的三轮校准（实测驱动）

分级公式是本项目迭代次数最多的部分，前后经过三轮实测校准，完整记录如下——这是一个"先实现、再测量、后修正"的典型工程过程。

第一轮问题：分级过于激进。

初版按线性面积占比评分，一个仅占图像面积 1% 的小缺陷在置信度 0.98 时也被判 D 级。

修正：面积项改为次线性变换 `sqrt(area / area_ref)`，压缩小尺度缺陷的贡献。

第二轮问题：面积项恒为满分。

进一步诊断（`src/diag_infer.py`）发现检测框实测面积占比为 **0.77~1.0**——滑窗分类只能给出「窗口内存在某类缺陷」，返回的框就是整个滑窗。此时面积项恒为满分，所有缺陷都被判 E 级。

修正：为面积项设置饱和上限 `area_cap = 0.30`。同时提供 `model.refine_topn` 选项，可对置信度最高的 N 个窗口用 Grad-CAM 反传收紧框边界（**默认关闭**：每窗口一次反向传播会把单图耗时从约 0.1 秒推到秒级，不适合现场演示）。

第三轮问题：等级退化，只有 A 和 D。（由实际演示中发现）

此时公式为 `100·(0.40·R + 0.25·C + 0.35·A)`，R 取 1.0/0.65/0.35。逐项拆解后定位到根因：**风险项自身就贡献 14~40 分，直接跨越多个等级区间**——

类型	风险项	置信项(×0.25)	面积项(×0.35)	总分	落档
高危(1.0)	40	23	≤10.5	~73.5	恒 D
中危(0.65)	26	23	≤10.5	~59.5	恒 C
低危(0.35)	14	23	≤10.5	~47.5	恒 C

中危与低危相差 12 分却只跨一档，B 级（28~42）几乎没有缺陷能落入。**根因是把"风险"当成了分数而不是分级先验。**

修正：改为**嵌套加权**（v2 公式），把风险降为占 `w_risk = 0.55` 的**基线先验**，其余 45% 权重交给「缺陷相对尺度 + 模型置信度」，从而形成"风险定基线、尺度与置信度决定升降级"的行为。

v2 实测效果（`python -m src.diag_grade` 与 `src/diag_infer.py` 输出）：

缺陷类型	风险档	实测等级	说明
龟裂、夹杂	高危	**D**	判废并停机排查
斑块、麻点、划痕	中危	**C**	转人工复检
压入氧化铁皮	低危	**B**	放行并记录
洁净样本	—	**A**	合格放行
（E 级保留给致命情形）			理论网格中不出现，作为保留档

理论网格覆盖统计（风险×置信度×面积 3×3×6 = 54 组合）：**B 19 例、C 23 例、D 12 例**，梯度充分。三级诊断脚本 `src/diag_grade.py`、`src/diag_infer.py`、`src/selftest_e2e.py` 随作品交付，可在其他产线上重新标定。

> **诚实说明**：受滑窗分类定位精度的限制，面积项在多数检测中仍会触及上限，因此**当前分级的主要区分依据是"缺陷类型风险 + 模型置信度"**，面积项起补偿与限幅作用。若需要让面积真正参与分级，可开启 `model.refine_topn` 获得更紧的缺陷边界，代价是单图耗时上升。

4.5 知识检索模块

`src/rag.py` 实现零依赖的中文检索：

- 分词：中文字符 2-gram 与 3-gram + 英文/数字按词切分；
- 打分：BM25（k1=1.5, b=0.75）逐项累加，IDF 平滑；
- 混合：`score = α · BM25_norm + (1-α) · n-gram 覆盖率`，α=0.65，弥补短查询下 BM25 失效；
- 字段加权：标题权重 2.0，体现"标准条款检索"的直觉。

知识库 `knowledge/quality_kb.json` 共 18 条专业条目，覆盖：六类缺陷的成因与工艺处置、严重度分级规则、可追溯性要求、人工复检规程、设备维护联动规程、历史案例、工艺环节对照表、系统适用边界、成本效益测算口径、政策依据。

4.6 智能体模块

`src/agent.py` 的 `QualityAgent` 提供五个工具并编排推理：

工具	作用
`retrieve_standard(query)`	检索标准与工艺知识，返回带分数的条目
`severity_grade(score)`	严重度分级
`create_work_order(det)`	按规则表生成工单（责任部门、检查项、是否停机、是否隔离批次）
`trace_batch(det)`	生成批次追溯建议
`self_check(result)`	置信度自检，判断是否需人工复检

推理流程：无缺陷则直接放行；有缺陷则按缺陷类型分别检索证据并去重合并，再用规则后端生成四段式建议（根因分析 / 立即处置 / 工艺纠正 / 批次追溯），若配置了本地或云端大模型后端则在规则结论基础上增强表达，最后调用自检与工单工具闭环。所有建议携带知识库条目 ID，实现结论可溯源。

4.7 接口与应用模块

`src/api.py` 提供 15 个 REST 接口（详见第 8 章接口清单）；`src/report.py` 支持 Markdown / HTML / JSON 三种报告格式，HTML 报告可直接在浏览器打印为 PDF，规避已归档的 wkhtmltopdf；`src/db.py` 负责检测记录、工单与检索留痕的持久化。

前端 `web/index.html` 为单文件原生实现，包含六个页签：智能检测、质量看板、检测记录、工单管理、知识库问答、系统说明。图像上传支持 `<input capture="environment">`，用手机浏览器打开即可调用摄像头拍照检测。

---

5 关键技术与实现

5.1 多尺度滑窗与 NMS 合并

工业缺陷尺度差异大：同一产线可能同时存在毫米级麻点与厘米级条带状压痕。单一窗口尺寸无法兼顾，因此采用 1.0 与 0.6 双尺度滑窗，窗口 224、步长 112（50% 重叠）。推理后按类别分别执行 NMS，IoU 阈值 0.35，避免同一缺陷被多个窗口重复计数。

5.2 界面设计与信息架构（Apple 设计语言）

前端经历了两轮调整，最终形成 5 个页签的结构。这里如实记录两轮的判断依据，因为它体现了一个常见的界面设计误区。

第一轮：功能平铺。

初版把系统所有能力平铺成六个页签（检测 / 看板 / 记录 / 工单 / 知识问答 / 系统说明），信息密度过高、主次不分。

第二轮：过度精简。

为追求"整洁"，把页签砍到 3 个（检测 / 看板 / 问答），并删掉了工单页、系统信息页、演示样本、热力图对比。结果**功能缺失**：闭环能力不可见、技术参数无处可查、可解释性无法现场演示。

最终结构（5 个页签）：判断标准不是"页面多少"，而是**每个页签是否对应演示主线上一个真实环节。**

页签	承载内容	对应演示环节
检测	上传/拍照、样本快选、结论、关键数字、四段式处置建议、缺陷明细、**原图/标注/热力图三联对照**、报告导出	感知与决策
看板	时间范围切换、6 项 KPI、等级分布、缺陷类型构成、逐小时趋势、**过程洞察**（把缺陷构成翻译成工艺排查方向）	质量管理
工单	工单列表、状态筛选、优先级/停机/隔离批次标记、检查项清单、状态流转	闭环执行
问答	自然语言问答 + 知识库原文检索双通道	知识沉淀
关于	技术参数、执行标准、严重度分级表、政策依据、技术路线时间轴、使用边界	答辩支撑

过程洞察是本轮新增的关键能力：光有数字不构成管理价值，系统会依据当前缺陷构成自动给出排查方向（如"划痕占比最高 → 优先排查导线接触部件与跑偏纠偏系统"），把看板从"展示数据"升级为"给出动作"。

视觉规范（`web/design-system.css`，约 400 行，按 token 组织）：

1. **内容优先但不阉割功能**：装饰做减法，能力做加法；
2. **层级靠尺寸与字重**：主标题 34px/700、卡片标题 17px/600、正文 15px/400、次要信息 13px/400 且降为次级色——苹果很少用颜色区分层级，主要靠字重与灰度；
3. **材质（Material）**：导航栏使用 `backdrop-filter: saturate(180%) blur(20px)` 毛玻璃 + 半透明底，滚动时内容从下方穿过；
4. **描边替代阴影**：默认只用 1px 细描边划分区块，避免页面出现大量"浮起来的盒子"；
5. **连续圆角**：卡片 18px、控件 10px、胶囊 980px；
6. **单一强调色**：仅用系统蓝 `#0071e3` 表达可交互与当前状态，其余走灰阶；
7. **页头呼吸感**：每个页签页头使用极淡径向渐变（`color-mix` 合成 9% 强调色），让长页面有层次而不花哨；
8. **深浅色自动适配**：通过 `prefers-color-scheme` 一次性适配，两个主题共用同一套变量。

组件库：分段控件（SegmentedControl）、标签组（Tag/Chip）、对比图（Compare）、样本条（Samples）、键值表（KV）、时间轴（Timeline）、工单卡（WorkOrder）、洞察卡（Insight）、统计行（Mini stats）。

工程约束：坚持**零外部依赖**（不引入任何 CDN 字体、图标库、前端框架），全部资源本地化，断网可完整渲染；样式与结构分离，`index.html` 内**不含任何内联 `<style>` 块。**

验证手段：新增 `src/check_frontend.py` 做静态自检——比对 HTML 中定义的 id 与 JS 引用的 id 是否一致（前端无构建链路，id 写错不会报错、只会静默失效）、校验页签与区块一一对应、确认无外部网络依赖。实测 39 个引用全部命中。

可解释性可视化：检测结果页提供「原图 / 缺陷标注 / Grad-CAM 热力图」三联对照（`src/viz_heatmap.py` 亦可批量生成专题图）。实测热力图高亮区与真实缺陷位置吻合——夹杂的块状、斑块的两个亮斑、麻点的点阵、氧化铁皮的条带均被准确定位，说明模型依据的是缺陷本身而非背景纹理，这是工业 AI 落地最需要排除的风险。

5.3 候选框融合与可视化实现

问题一：同一缺陷被重复框出多遍。

滑窗步长 112、窗口 224（50% 重叠），且使用双尺度，实测一张图上同一处缺陷会产生 5~7 个高度重叠的框，前端画面杂乱、严重度也无法解释。仅靠 NMS 无法解决——不同尺度的窗口大小差异大，IoU 很低但显然是同一缺陷。

修正：在 NMS 之前增加一层**候选融合**（`_merge_candidates`）：

1. 用"交叠面积 / 较小框面积"代替 IoU 判定同一性（IoU 会被大并集稀释，导致该合并的没合并）；
2. 逐类别、按置信度从高到低贪心分组；
3. **并集膨胀约束**：合并后并集面积不得超过合并前较大框面积的 1.6 倍，否则视为不同位置的缺陷而不合并。若缺此约束，"链式重叠"会一路把框并到覆盖整张图——开发中实测出现过 256×256 的全图框；
4. 组内取最高置信度，坐标取组内并集。

问题二：中文标签显示为乱码。

OpenCV 的 `putText` 只支持 Hershey 矢量字体，**无法渲染中文**，实测图上标签显示为 `?????? 0.98 D`。

修正：标签改由 PIL 绘制，按平台常见路径依次探测中文字体（Windows 的 msyh/simhei/simsun、Linux 的 wqy/Noto CJK、macOS 的 PingFang），并加入标签纵向避让，避免多个框的标签互相压盖。相关可视化核验脚本为 `src/visualize_result.py`（输出"原图 / 标注图 / Grad-CAM 热力图"三联图），可直接用于人工复查与文档插图。

5.4 置信度校准与人工复检策略

系统计算 ECE（期望校准误差）评估置信度可靠性。当置信度低于 0.7 时，`self_check` 工具会将结果标记为"需人工复检"，而不是强行判定。这一设计把模型的"不确定"显式暴露给质检员，是工业场景下比单纯追求准确率更重要的工程取舍。

5.5 三级推理后端与降级

```
backend_priority: [rule, ollama, openai_compat]
```

- `rule`：确定性规则 + 知识库抽取，永远可用，可离线复现；
- `ollama`：本地大模型（如 Qwen3 系列），数据不出厂；
- `openai_compat`：任意 OpenAI 兼容云 API，用于演示更强推理能力。

降级时在结果中记录 `degraded_reason`，并在报告与前端展示，保证过程透明。

> 许可说明：本项目在文档中不推荐使用 Qwen2.5-VL-3B-Instruct 等采用 `qwen-research`（仅限研究、不可商用）许可的模型作为企业交付卖点；本地后端建议选用 Apache-2.0 或 MIT 许可的模型。

5.6 数据与结果的完整留痕

每次检测记录：检测编号、时间、批次、原始图像路径、图像尺寸、滑窗数量、全部缺陷坐标与置信度、严重度、判定结论、耗时、智能体建议与引用条目。满足 GB/T 40659-2021 对在线检测系统可追溯性的要求，也便于赛后复盘与模型迭代。

6 数据库设计

6.1 E-R 关系

```
inspections (检测记录) 1 ——— n work_orders (工单)
        |
        └——— n kb_queries (检索留痕)
```

6.2 表结构

inspections（检测记录）

字段	类型	说明
id	INTEGER PK	自增主键
inspection_id	TEXT UNIQUE	检测编号，如 QC20261012223001123
created_at	TEXT	检测时间
image_path / image_name	TEXT	图像路径与文件名
image_size	TEXT	图像尺寸 JSON
verdict	TEXT	合格 / 不合格
grade / grade_zh	TEXT	严重度等级与中文名
max_severity	REAL	最高严重度得分
defect_count	INTEGER	检出缺陷数
top_defect_code / top_defect_zh	TEXT	主要缺陷
confidence	REAL	主要缺陷置信度
elapsed_ms / windows	REAL / INTEGER	耗时与滑窗数
payload	TEXT	完整结构化结果 JSON
advice	TEXT	智能体建议 JSON
batch_no / operator	TEXT	批次与检测终端

work_orders（工单）：work_order_id、inspection_id、created_at、defect_code、defect_zh、severity_level、severity_score、priority、department、action、check_items、stop_line、quarantine_batch、status。

kb_queries（检索留痕）：created_at、query、hit_ids、backend。

7 系统测试与指标

7.1 测试环境

项目	配置
操作系统	Windows（沙箱/受限环境亦可运行）
CPU	无独立显卡，纯 CPU 训练与推理
Python	3.10
框架	PyTorch 2.12（CPU 版）、torchvision 0.27、OpenCV 4.11、FastAPI
模型主干	MobileNetV3-Small，1.08 M 参数

7.2 数据集

原始数据由 `src/data_generate.py` 程序化合成，共 **1400 张** 256×256 灰度图像，7 类（crazing / inclusion / patches / pitted_surface / rolled_in_scale / scratches / clean）各 200 张，按 7:1.5:1.5 分层抽样划分为训练 980 / 验证 210 / 测试 210 张。

真实公开数据接入通道已实现并验证连通（GC10-DET / DeepPCB / KolektorSDD2 / VisA），其中部分镜像在受限网络下不稳定，系统支持将真实数据放入 `data/real/` 或 `data/custom/` 目录后自动识别并混入训练，形成"合成预训练 + 真实微调"的迁移路线。

> **数据来源说明（重要）**：本章指标基于程序化合成数据集的留出测试集，用于验证系统完整性与算法有效性；合成数据不能替代真实产线数据，实际部署须以企业自有数据微调后的指标为准。系统不对此作夸大表述。

7.3 检测性能指标

<!-- METRICS_TABLE_START -->

测试环境：cpu（无独立显卡） | 主干：mobilenetv3_small_100 | 参数量：1.077 M | 实际训练 0 轮，最佳第 5 轮，用时 0.0 秒

总体指标

指标	验证集	测试集
准确率 Accuracy	100.00%	100.00%
Top-2 准确率	100.00%	100.00%
宏平均 F1	100.00%	100.00%
加权 F1	100.00%	100.00%
ECE（期望校准误差）	0.0521	0.0497
单图分类耗时	9.28 ms	9.92 ms

逐类别指标（测试集）

缺陷类别	样本数	精确率 P	召回率 R	F1
龟裂（`crazing`）	30	100.00%	100.00%	100.00%
夹杂（`inclusion`）	30	100.00%	100.00%	100.00%
斑块（`patches`）	30	100.00%	100.00%	100.00%
麻点（`pitted_surface`）	30	100.00%	100.00%	100.00%
压入氧化铁皮（`rolled_in_scale`）	30	100.00%	100.00%	100.00%
划痕（`scratches`）	30	100.00%	100.00%	100.00%
洁净(合格)（`clean`）	30	100.00%	100.00%	100.00%

鲁棒性评估（受控退化实验，压力集不参与训练）

难度档位	样本数	检出率	漏检率	过杀率	缺陷分类准确率
L1 正常（与训练同分布）	210	100.00%	0.00%	0.00%	99.44%
L2 小尺度（缺陷尺寸约 45%）	210	100.00%	0.00%	0.00%	97.78%
L3 小尺度+低对比度（对比度约 40%）	210	85.00%	15.00%	0.00%	79.44%

常规尺度与小尺度下检出率 100%、过杀率 0%；**小尺度叠加低对比度时漏检率升至 15%**，量化了「缺陷过小且对比度不足」这一主要失效模式。

训练曲线与混淆矩阵见 `runs/figures/training_curves.png`、`runs/figures/confusion_matrix.png`；完整指标报告见 `runs/METRICS.md`；鲁棒性原始数据见 `runs/robustness.json`。

> 以上指标基于程序化合成数据集的留出测试集与压力测试集，用于验证系统完整性与算法有效性。真实公开数据集（GC10-DET / DeepPCB / KolektorSDD2 / VisA）接入通道已实现，将数据放入 `data/real/` 或 `data/custom/` 后重建索引即可混入训练。

<!-- METRICS_TABLE_END -->

7.4 训练过程

训练曲线与混淆矩阵见 `runs/figures/training_curves.png` 与 `runs/figures/confusion_matrix.png`。

7.5 功能测试

编号	测试项	步骤	预期结果
T01	图像上传检测	选择样本图片点击检测	返回缺陷框、等级与建议动作
T02	手机拍照检测	手机浏览器打开页面，调用摄像头拍照	同上，链路一致
T03	严重度分级	输入不同面积/类型的缺陷	等级随风险与面积单调变化
T04	智能体建议	检测含缺陷图像	输出四段式建议并附知识库引用
T05	工单生成	完成一次 D/E 级检测	自动生成高优先级工单，标记停机与隔离批次
T06	报告导出	点击报告链接	浏览器打开 HTML 报告，可打印为 PDF
T07	看板统计	填充演示数据后查看看板	合格率、等级分布、类型分布、趋势正确
T08	知识问答	提问"龟裂成因与处置"	返回可溯源答案并列出引用条目
T09	后端降级	关闭本地大模型	自动降级为规则后端并记录原因
T10	无缺陷图像	上传洁净样本	判定合格并放行

7.6 鲁棒性与边界测试

合成数据生成器支持对缺陷的**尺度、对比度、密度**参数化，用于压力测试。测试结论同时写入知识库条目 `TECH-LIMITS`：缺陷与背景对比度极低且尺度小于成像分辨率时存在漏检风险；强反光、油污、水渍会造成误报；未参与训练的缺陷类型会被归入最相似的已知类别，须依赖持续标注迭代或引入无监督异常检测补充。

---

8 部署与操作说明

8.1 环境要求

Python 3.10+，依赖：torch、torchvision、opencv-python、numpy、fastapi、uvicorn、python-multipart、matplotlib（绘图可选）。

8.2 安装与启动

```
pip install torch torchvision opencv-python numpy fastapi uvicorn python-multipart matplotlib pyyaml


python -m src.data_generate --per-class 200   # 1. 生成合成数据集
python -m src.data                            # 2. 构建索引与划分
python -m src.train --epochs 26               # 3. 训练模型
python -m src.api                             # 4. 启动服务
# 浏览器打开 http://127.0.0.1:8077
```

8.3 REST 接口清单

方法	路径	说明		
GET	`/api/health`	健康检查与系统状态		
GET	`/api/system`	系统信息、政策与标准引用		
GET	`/api/classes`	缺陷类别字典		
POST	`/api/inspect`	上传图像检测（可选热力图与智能体）		
POST	`/api/inspect/agent`	检测 + 智能体处置一体化		
GET	`/api/inspections`	检测记录列表（支持筛选）		
GET	`/api/inspections/{id}`	单条记录详情		
GET	`/api/dashboard`	看板统计		
GET	`/api/work_orders`	工单列表		
POST	`/api/work_orders/{id}/status`	更新工单状态		
POST	`/api/kb/search`	知识库检索		
POST	`/api/chat`	质检知识问答		
GET	`/api/report/{id}?fmt=html\`	md\	json`	生成质检报告
GET	`/api/samples`	演示样本列表		
POST	`/api/demo/seed`	填充演示数据		

8.4 现场演示步骤（建议）

1. 打开首页，展示健康状态（模型已就绪、知识库条目数）；
2. 点击左侧"压入氧化铁皮"样本 → 展示检测框、等级 B、处置建议；
3. 点击"龟裂"样本 → 展示等级 D，自动生成紧急工单并标记停机与隔离批次；
4. 切换到 Grad-CAM 热力图页签 → 说明模型关注区域，体现可解释性；
5. 打开质检报告 → 展示完整留痕，点击打印说明可导出 PDF；
6. 切换到质量看板 → 展示合格率、缺陷分布与趋势；
7. 知识库问答输入"龟裂缺陷的成因和工艺处置措施是什么" → 展示可溯源答案；
8. 手机扫码/输入局域网地址，现场拍照检测，说明落地形态。

---

9 创新点与技术特色

1. **从"检测"到"处置"的闭环**：现有低成本方案止步于缺陷识别，本作品用智能体补齐根因分析、工艺纠正、工单联动与批次追溯，形成"感知—决策—执行"闭环。

2. **确定性判定 + 大模型增强的混合架构**：质量判定由规则与阈值完成，保证可复现、可审计；大模型只做解释与建议生成，规避幻觉影响质量判定。
3. **零额外依赖的核心链路**：检索、可解释性、报告生成均自研实现，演示环境无网络、无法安装第三方包时仍能完整运行。
4. **面向小样本冷启动的数据策略**：程序化合成缺陷 + 公开真实数据 + 企业自有数据三层递进，并支持参数化压力测试，直接回应政策文件提出的"小样本标注缺陷数据微调"。
5. **许可干净的工程选型**：检测主干采用 BSD-3 许可的 torchvision 主干而非 AGPL 的 YOLO 系，便于企业交付；文档中对各模型许可风险逐项说明。
6. **置信度校准驱动的复检策略**：计算 ECE 并把低置信度结果显式转为人工复检，而非强行判定，符合工业质量管理的实际逻辑。

---

10 开发规范与工具说明

10.1 代码规范

- 全中文注释与文档字符串，便于评审阅读；
- 模块职责单一，配置集中管理；
- 所有脚本提供 `--help` 与命令行参数；
- 关键算法（严重度评分、BM25、NMS、Grad-CAM）均给出公式与实现说明。

10.2 AI 辅助编程使用情况说明

本项目开发过程中使用了 AI 辅助编程工具生成部分样板代码与文档草稿。所有生成内容均经人工逐行审阅、调试与验证，核心算法与设计决策由团队确定。开发过程记录了两个由 AI 生成代码引入、后经自检脚本发现的 `torchvision` 使用缺陷（详见 4.2 节），说明工程验证环节并未被省略。

10.3 第三方资源与许可

资源	许可	用途
PyTorch / torchvision	BSD-3-Clause	深度学习框架与主干网络
OpenCV	Apache-2.0	图像处理
FastAPI / Uvicorn	MIT / BSD	Web 服务
MobileNetV3 预训练权重（ImageNet）	BSD-3-Clause	迁移学习初始化
公开缺陷数据集（GC10-DET / DeepPCB / KolektorSDD2 / VisA）	各自许可（多为仅研究用途）	方法验证；实际部署须改用企业自有数据
大模型后端（可选）	各自许可	建议选用 Apache-2.0 / MIT 许可模型

附录 A 项目目录结构

```
bisai/
├── config/config.yaml              全局配置
├── knowledge/quality_kb.json       质检标准知识库（18 条）
├── src/
│   ├── utils.py                    配置、日志、路径、分级工具
│   ├── data_generate.py            合成缺陷数据集生成器
│   ├── data_real.py                真实数据集下载整理（KolektorSDD2 / VisA）
│   ├── data_github.py              GitHub 归档型数据集（GC10-DET / DeepPCB）
│   ├── data.py                     数据索引、划分、Dataset
│   ├── model.py                    轻量 CNN + Grad-CAM
│   ├── train.py                    训练与评估
│   ├── infer.py                    滑窗检测 + NMS + 分级 + 可视化
│   ├── rag.py                      BM25 + n-gram 混合检索
│   ├── agent.py                    质检处置智能体
│   ├── report.py                   报告生成（MD/HTML/JSON）
│   ├── db.py                       SQLite 持久化
│   ├── api.py                      FastAPI 后端
│   ├── fetch_util.py               断点续传下载器与归档检查
│   ├── fetch_data.py               多镜像下载探针
│   ├── inspect_repo.py             仓库归档内容检查
│   ├── diag_net.py                 网络通道诊断
│   ├── visualize_data.py           数据集抽样可视化
│   └── selfcheck_model.py          主干与 Grad-CAM 自检
├── web/index.html                  单文件前端
├── runs/                           训练产物（指标、图表、索引）
├── models/                         模型权重
└── data/                           数据集、样本、数据库、报告
```

附录 B 术语表

术语	说明
滑窗（Sliding Window）	用固定大小窗口在图像上按步长滑动切块，逐块分类实现定位
NMS	非极大值抑制，合并重叠检测框
Grad-CAM	基于梯度的类激活映射，用于可视化模型关注区域
ECE	期望校准误差，衡量预测置信度与实际准确率的一致性
BM25	经典概率检索模型，含词频饱和与文档长度归一化
RAG	检索增强生成，先检索知识再生成回答
严重度分级	按缺陷风险、置信度与面积综合评定 A~E 五个等级

文档结束。本作品判定结论与处置建议仅供工艺参考，最终处置须由有资质人员确认。